AlphaStream India

Real-Time AI Investment Intelligence
for the Indian Retail Investor

13	<2s	30+	5	14Cr+
AI Agents	News-to-Signal	API Endpoints	Tab Terminal	Target Users

Layer 1: Indian & Global Data Sources
NSE API • BSE API • FII/DII (NSDL) • Groww API • ET Markets RSS
WorldMonitor • FRED Macro • CNN Fear & Greed • NewsAPI

↓

Layer 2: Pathway Streaming Engine
ConnectorSubject • Adaptive RAG • pw.io.subscribe • DocumentStore

↓

Layer 3: Multi-Agent Reasoning (13 Agents)
Sentiment • Technical • Risk • Pattern • Backtest • Flow • Filing • Insider • Anomaly • Search • Chart • Report • Decision

↓

Layer 4: DuckDB Analytics + NLQ Engine
DuckDB • ChromaDB • Text2SQL • LangGraph 8-Node Pipeline • MCP Servers

↓

Layer 5: Bloomberg-Style React Terminal (5 Tabs)
Overview • Signals • Global Intel • Company • Portfolio

Competition: ET AI Hackathon 2026 – Avataar.ai + Unstop
Problem: PS-6: AI for the Indian Investor
Repository: https://github.com/wildcraft958/AlphaStream_India
Date: March 2026

Contents

1 Executive Summary **2**

2 Problem Statement **3**
2.1 The Scale of the Problem . . . 3
2.2 Our Solution: Live AI . . . 3

3 System Architecture **4**
3.1 Five-Layer Architecture . . . 4
3.2 End-to-End Data Flow . . . 5
3.3 DuckDB Analytics Schema . . . 5
3.4 Fusion Engine – Alpha Score . . . 5

4 Pathway Streaming Integration **6**
4.1 Pathway Features Used . . . 6
4.2 Adaptive RAG – Geometric Retrieval . . . 6
4.3 Fallback Strategy . . . 7

5 Multi-Agent Reasoning System **7**
5.1 Overview . . . 7
5.2 Agent Architecture . . . 8
5.3 Agent Specifications . . . 8
5.4 Signal Types Detected . . . 9
5.5 Error Handling . . . 9

6 NLQ Agent Pipeline **9**
6.1 LangGraph 8-Node Pipeline . . . 9
6.2 MCP Servers (Model Context Protocol) . . . 10

7 WorldMonitor Global Intelligence **10**
7.1 Global Data Feeds . . . 10
7.2 Integration Architecture . . . 11

8 Technology Stack **11**
8.1 Backend . . . 12
8.2 Frontend . . . 12

9 API Reference **12**
9.1 Market Intelligence (15 endpoints) . . . 12
9.2 Global Market (9 endpoints) . . . 13
9.3 Core + Streaming (6 endpoints) . . . 13

10 Bloomberg-Style Terminal (Frontend) **13**

11 Business Impact Model **14**
11.1 Market Sizing (TAM / SAM / SOM) . . . 15

11.2 User Impact – Time Saved 15
11.3 User Impact – Alpha Generated 15
11.4 Risk Reduction 15
11.5 ET Markets Revenue Projection 15
11.6 Cost Reduction for ET Markets 16
11.7 Competitive Moat 16
11.8 Assumptions (Stated Explicitly) 16

12 Demonstration & Verification **16**
12.1 Live Demo Flow 16
12.2 Expected Latencies 17

13 Setup & Quick Start **17**
13.1 Prerequisites 17
13.2 Three-Command Setup 17
13.3 Key Environment Variables 17

1. Executive Summary

One-Line Pitch

AlphaStream India is a Bloomberg-style AI investment terminal that transforms ET Markets data into actionable, real-time, backtested signals for India's **14 crore+** retail investors – replacing 2+ hours of daily research with AI-powered intelligence delivered in under 2 seconds.

India has over **14 crore demat accounts** (CDSL + NSDL, 2024), yet the vast majority of retail investors are flying blind – reacting to WhatsApp tips, unable to read technicals, missing corporate filings, and managing portfolios on gut feel. Institutional investors use Bloomberg terminals costing $25,000/year. AlphaStream India bridges this gap with a **free, production-grade** alternative built on cutting-edge AI.

What We Built

- **Opportunity Radar** – continuously monitors corporate filings, insider trades, bulk/block deals, FII/DII flows, and regulatory changes. Surfaces missed opportunities as daily alerts with Alpha Scores (0–100). Not a summarizer – a *signal finder*.
- **Chart Pattern Intelligence** – real-time detection of breakouts, RSI divergences, MACD crossovers, Bollinger breakouts, golden/death crosses across the entire Nifty 50 universe. Every pattern is backtested over 5 years with verifiable win rates.
- **Market ChatGPT – Next Gen** – deeper data integration via Text2SQL against a live DuckDB analytics database. Portfolio-aware, multi-step analysis, source-cited responses. Answers grounded in *real data*, not LLM hallucination.
- **WorldMonitor Global Intelligence** – live global indices, crypto, commodities, FX, Fear & Greed, macro signals, and geopolitical risk – all wired into every recommendation with India-specific impact analysis.

Key Differentiators

What 99% of Teams Build	What AlphaStream Builds
Call GPT-4 → get text blob	Text2SQL pipeline → grounded in live DuckDB data
No verification of claims	Historical 5-year backtest with verifiable win rates
Mock or delayed data	Live NSE / BSE / FII / NSDL data feeds
Streamlit prototype	Production React 19 + WebSocket + SSE terminal
Single ticker lookup	13-agent Alpha Score fusion across all Nifty 50
No guardrails	Input/output guardrails + SQL correction loop
Generic global tickers	India-first: Rs. , IST, NSE/BSE, FII/DII, BSE filings
Batch processing	Pathway streaming: news to signal in <2 seconds

Submission Artefacts

GitHub Repository	https://github.com/wildcraft958/AlphaStream_India
Architecture Doc	Separate PDF: Architecture + Impact Model
Impact Model	Section 11 of this document + separate PDF
Demo Video	3-minute walkthrough (see script in docs/)

2. Problem Statement

2.1 The Scale of the Problem

14 Cr+	80%	2+ hrs	Rs. 10K	0
Demat Accounts	Rely on Tips	Daily Research	Missed/Quarter	Free AI Terminals

India has **14 crore+ demat accounts** yet the tools available to retail investors are dramatically inferior to institutional systems:

- **Stale data** – most free platforms update prices every 5–15 minutes; institutional desks consume tick data in real time.
- **No signal aggregation** – a retail investor checks RSI on one screen, reads FII data on NSDL's PDF portal, hunts for insider trades on NSE's filing search – three separate workflows with zero synthesis.
- **LLM tools are ungrounded** – existing "AI chatbots" for investing answer questions from trained weights, not live market data. They hallucinate stock prices and miss earnings announced an hour ago.
- **2+ hours of daily research** – active retail investors spend 2–3 hours/day on research that could be automated, translating to **Rs. 2.19 Lakh/year** in lost productivity per user.
- **No free alternative to Bloomberg** – institutional terminals cost $25,000+/year. Retail investors have nothing comparable.

2.2 Our Solution: Live AI

AlphaStream implements the **"Live AI" paradigm** – a system that continuously ingests data, updates its knowledge base incrementally (not batch), delivers recommendations reflecting the current state of reality, and provides full explainability for every decision.

Traditional AI Tool	AlphaStream India
Call GPT-4 → text blob	Text2SQL → grounded DuckDB answer
No verification	Historical backtest with win rates
Mock / delayed data	Live NSE / BSE / FII data
Streamlit dashboard	Production React + WebSocket
Single ticker lookup	13-signal Alpha Score fusion
No guardrails	Input/output guardrails + SQL correction loop
Generic (global tickers)	India-first (NSE/BSE, Rs. , IST, FII/DII)

3. System Architecture

3.1 Five-Layer Architecture

The system is organized into five vertically-stacked layers, each communicating with the next via well-defined interfaces: REST calls, Pathway streaming callbacks, WebSocket events, and DuckDB SQL queries.

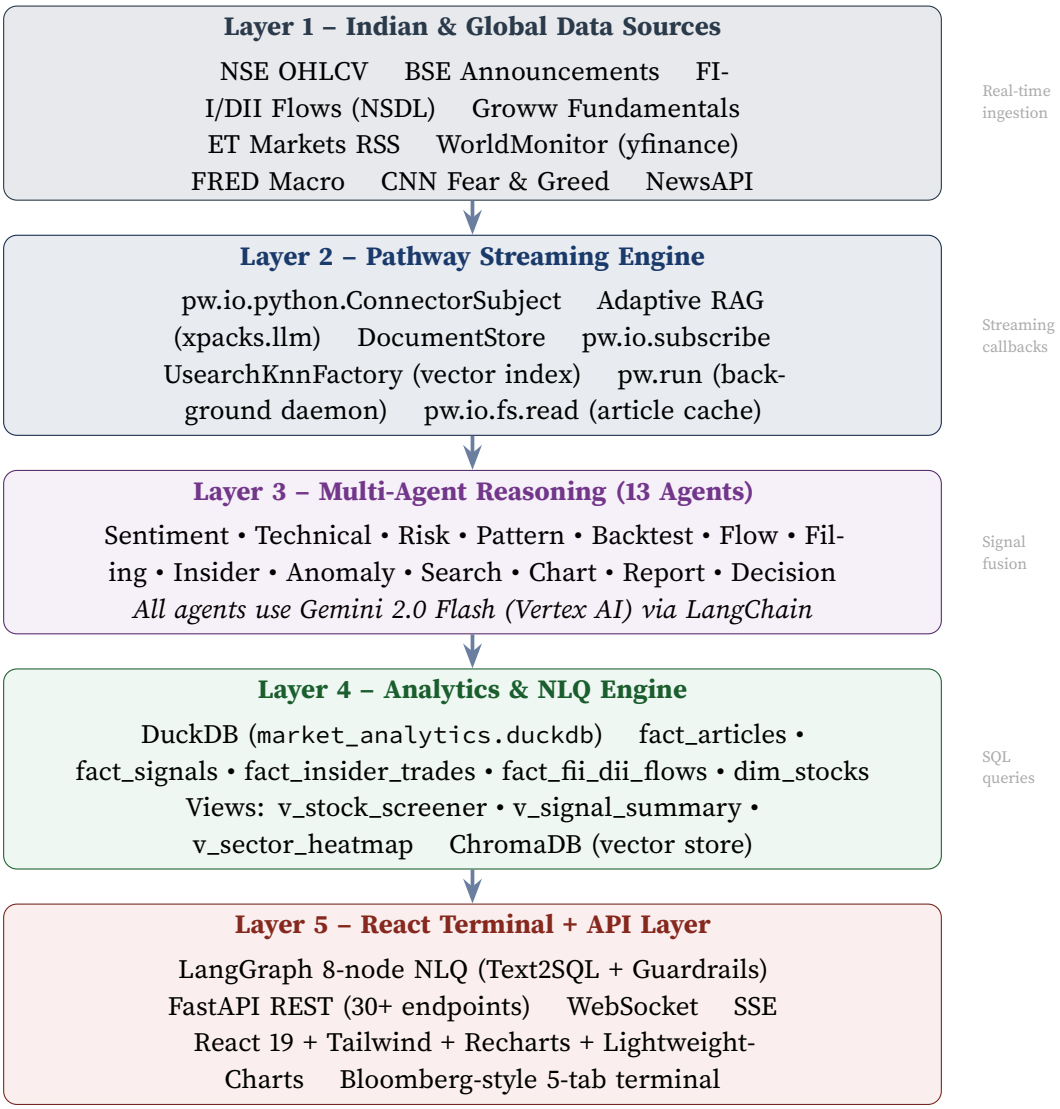


Figure 1: AlphaStream India – Five-Layer System Architecture

3.2 End-to-End Data Flow

1. **Ingest**: The Pathway `ConnectorSubject` polls 5 news APIs in parallel every 60 seconds; articles are written to `data/articles/`. NSE/BSE connectors fetch OHLCV, filings, and insider trades.

2. **Index**: `pw.io.fs.read` detects new files and the `DocumentStore` chunks, embeds (all-MiniLM-L6-v2), and indexes them into UsearchKnnFactory – all incrementally, with no full rebuild.

3. **Callback**: `pw.io.subscribe` fires `on_new_article()` on each new row, triggering DuckDB persistence and WebSocket broadcasts.

4. **Reason**: On a `POST /recommend` request, 13 specialist agents run in parallel (via `asyncio.gather`). The FusionEngine combines scores with configurable weights into a single Alpha Score (0–100).

5. **Deliver**: The response is pushed to the React dashboard over WebSocket (`/ws/stream/{ticker}`) in under 50 ms.

3.3 DuckDB Analytics Schema

The analytics layer uses a star schema optimized for fast aggregation:

Table / View	Purpose
`dim_stocks`	Nifty 50/200 universe: ticker, name, sector, ISIN, market cap
`fact_daily_prices`	OHLCV for all stocks (ticker + date PK)
`fact_signals`	Detected signals with alpha_score, evidence_json, backtest_json
`fact_insider_trades`	NSE SAST/PIT data: person, trade type, quantity, date
`fact_fii_dii_flows`	Daily institutional flows: FII buy/sell, DII buy/sell
`fact_filings`	BSE corporate announcements: results, dividends, board meetings
`fact_articles`	Ingested news with tickers, sentiment, threat level
`v_stock_screener`	Nifty 50 with latest price, active signals, insider activity
`v_signal_summary`	Recent signals grouped by ticker/type/date
`v_insider_activity_30d`	Rolling 30-day insider trades by sector
`v_fii_dii_trend`	FII/DII streaks (5+ consecutive days)
`v_sector_heatmap`	Sector-level signal counts & average alpha scores
`v_recent_news`	Articles from last 7 days with sentiment badges

3.4 Fusion Engine – Alpha Score

The FusionEngine produces a single **Alpha Score (0–100)** for each ticker by combining all agent outputs with configurable weights:

Signal Source	Weight	What It Captures
Filing Agent	0.25	Material BSE announcements (results, dividends)
Technical Agent	0.25	RSI, MACD, SMA trend direction
Insider + Flow	0.20	FII/DII streaks + insider cluster buying
Sentiment Agent	0.15	News sentiment across multiple sources
Backtest Agent	0.15	Historical validation of detected patterns

Each component score is normalized to 0–100. The weighted sum maps to a direction: **STRONG_BUY** ($\geq$75), **BUY** (60–74), **HOLD** (40–59), **SELL** (25–39), **STRONG_SELL** ($<$25).

4. Pathway Streaming Integration

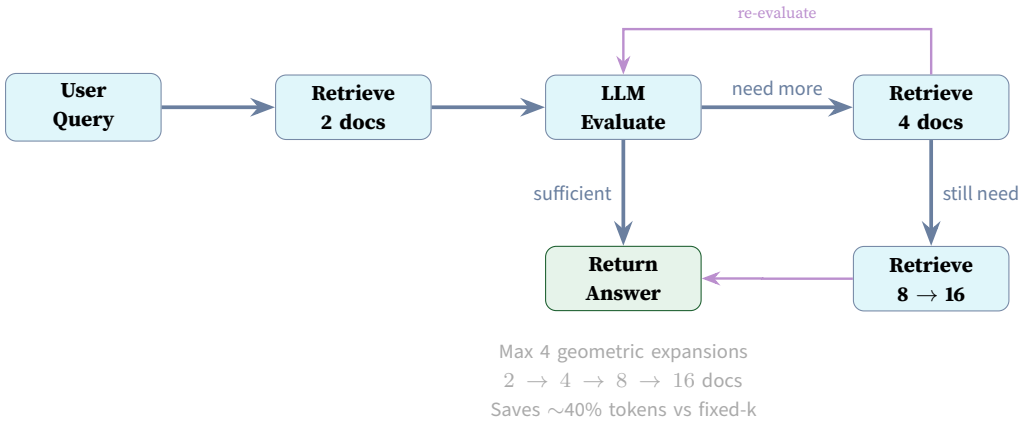
AlphaStream uses Pathway as its **primary real-time engine** – not as a library bolt-on but as the backbone of the entire ingestion and retrieval pipeline.

4.1 Pathway Features Used

Pathway API	File	Purpose
`pw.Schema`	`news_connector.py`	Type-safe streaming schema
`pw.Table`	`pathway_tables.py`	Streaming market data tables
`pw.io.python .ConnectorSubject`	`news_connector.py`	Custom polling connector
`pw.io.fs.read`	`adaptive_rag_server.py`	Auto-detect new articles
`pw.io.subscribe`	`app.py`	Real-time callbacks
`pw.run`	`app.py`	Background engine thread
`pw.apply`	`pathway_tables.py`	UDF transformations
`pw.filter`	`pathway_tables.py`	Event filtering
`pw.reducers`	`pathway_tables.py`	Aggregations (avg, count, max)
`pw.indexing .UsearchKnnFactory`	`adaptive_rag_server.py`	Vector similarity search
`pw.xpacks.llm.*`	`adaptive_rag_server.py`	Official LLM components
`AdaptiveRAG QuestionAnswerer`	`adaptive_rag_server.py`	Geometric retrieval
`pw.persistence`	`pathway_rag.yaml`	Caching & fault tolerance

4.2 Adaptive RAG – Geometric Retrieval

The Adaptive RAG starts with a minimal document set and expands geometrically only when the LLM signals insufficient context, saving $\sim$**40% tokens**:



Listing 1: Adaptive RAG Configuration

```
from pathway.xpacks.llm.question_answering import AdaptiveRAGQuestionAnswerer
from pathway.xpacks.llm.document_store import DocumentStore

document_store = DocumentStore(
    docs=pw.io.fs.read(path="data/articles", format="binary"),
    splitter=splitters.TokenCountSplitter(max_tokens=400),
    retriever_factory=pw.indexing.UsearchKnnFactory(
        embedder=embedders.SentenceTransformerEmbedder("all-MiniLM-L6-v2"),
        metric=pw.indexing.USearchMetricKind.COS))

question_answerer = AdaptiveRAGQuestionAnswerer(
    llm=llms.LiteLLMChat(model="openrouter/google/gemma-3n-e2b-it:free"),
    indexer=document_store,
    n_starting_documents=2, factor=2, max_iterations=4)
```

4.3 Fallback Strategy

When `ENABLE_PATHWAY=false` (default for quick setup), the system uses a synchronous `RAGPipeline` backed by ChromaDB with in-memory embeddings. All API endpoints behave identically – only the latency profile changes.

5. Multi-Agent Reasoning System

5.1 Overview

Every recommendation is produced by **13 specialist agents** running in parallel. Each agent is a LangChain chain with a narrow, well-defined responsibility. The Decision Agent receives all outputs and synthesizes a final signal.

5.2 Agent Architecture

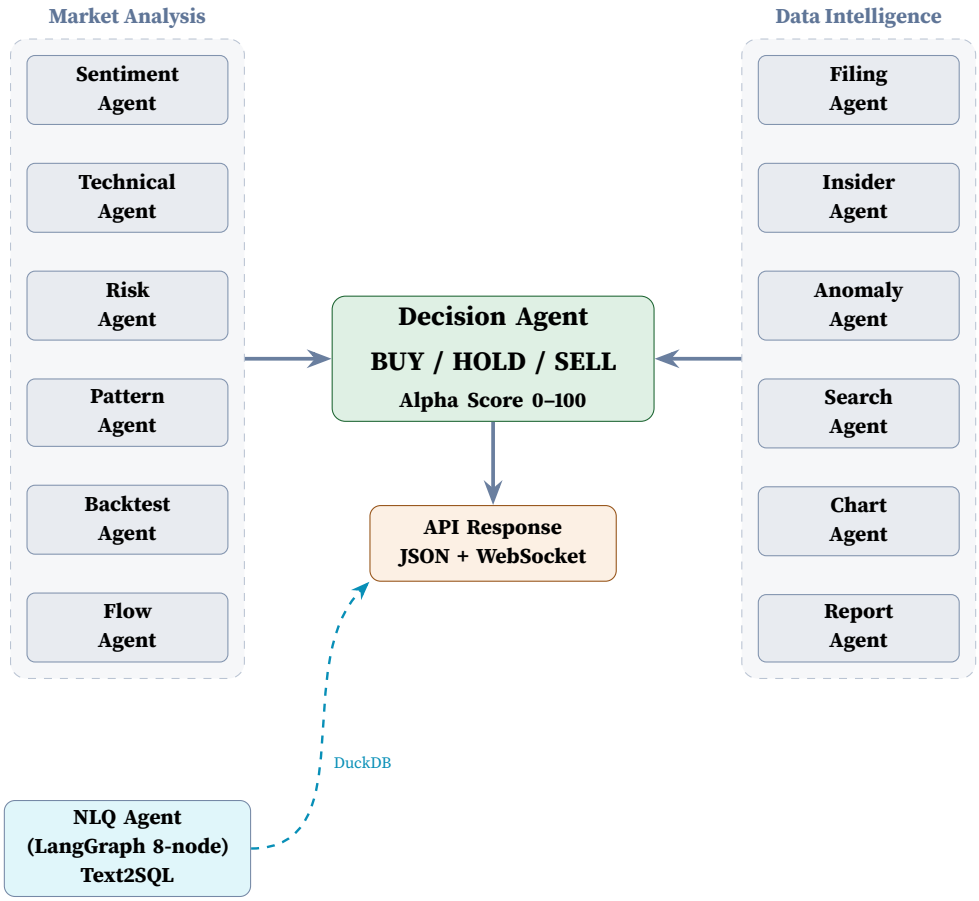


Figure 2: 13-Agent Multi-Agent Reasoning System with Alpha Score Fusion

5.3 Agent Specifications

Agent	Input	Output	Technology	Data Source
Sentiment	News articles	Score (-1 to $+1$), label	LangChain + Gemini	NewsAPI, RSS
Technical	NSE OHLCV	RSI, SMA trend signals	yfinance + numpy	NSE API
Risk	Price data	Volatility, stop-loss	Statistical calc	yfinance
Pattern	OHLCV (6 mo)	7 chart patterns	Custom detectors	yfinance
Backtest	Ticker + pattern	Win rate, Sharpe (5yr)	yfinance + stats	Historical
Flow	FII/DII data	Net flow, streak detect	NSDL + analysis	NSDL
Filing	BSE annc.	Materiality, impact	BSE API + Gemini	BSE API
Insider	SAST/PIT data	Cluster buy/sell	NSE + Gemini	NSE SAST
Anomaly	OHLCV series	Anomaly flags + scores	River ML (HST)	yfinance
Search	NLQ query	Web search results	LangGraph cyclic	DuckDuckGo
Chart	Ticker + events	PNG chart image	Matplotlib	yfinance
Report	All outputs	PDF report	ReportLab	All agents
Decision	All agents	BUY/HOLD/SELL + Alpha	LangChain + Gemini	All agents

5.4 Signal Types Detected

Signal	Detection Method	Win Rate	Avg 30d Return
RSI Divergence	Price/RSI divergence scan	60–78%	+3.26%
MACD Crossover	Signal line cross + histogram	45–57%	+0.71%
Bollinger Breakout	Squeeze → expansion	55–65%	+2.1%
Volume Breakout	>2x 20-day average	50–60%	+1.8%
Golden Cross	50 SMA > 200 SMA	65–75%	+4.2%
Insider Cluster	3+ insiders in 30 days	High	Varies
FII Buying Streak	5+ consecutive sessions	∼72%	+3–5%

5.5 Error Handling

Failure Scenario	Recovery Strategy
Individual agent exception	Caught; score defaults to 0; other agents continue
LLM timeout / API error	Exponential backoff × 3; falls back to rule-based score
Pathway callback error	try/except in `on_new_article`; logs ERROR; Pathway continues
DuckDB write contention	`threading.Lock` guards all writes
NSE API blocked	yfinance `.NS` suffix as fallback
MCP server crash	Graceful degradation – direct DuckDB queries
WebSocket disconnect	`ConnectionManager.disconnect()` removes client
Missing GCP credentials	Pydantic `model_validator` warns at startup

6. NLQ Agent Pipeline

The NLQ (Natural Language Query) system converts free-form investor questions into SQL queries against the live DuckDB analytics database, returning **grounded, source-cited answers**.

6.1 LangGraph 8-Node Pipeline

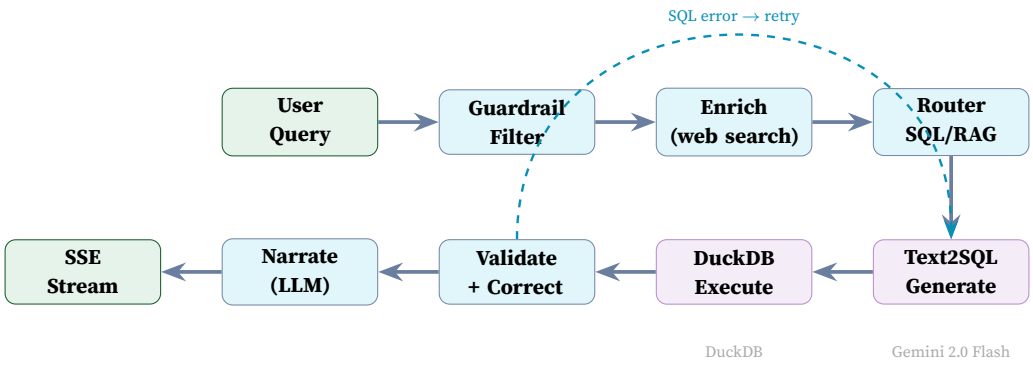


Figure 3: LangGraph 8-Node NLQ Pipeline with SQL Correction Loop

#	Node	Responsibility
1	Input Guardrail	Rejects off-topic/harmful queries; detects PII (emails, phones), injection patterns
2	Enrichment	Optional web search for context not in DB (recent events, company background)
3	Router	Classifies: analytics / kpi_lookup / portfolio / off_topic
4	Text2SQL	Schema linking → query planning → SQL generation via Gemini 2.0 Flash
5	DuckDB Execute	Runs query; captures result set or error (30s timeout, 5000-row cap)
6	Validate + Correct	Checks for empty sets, type errors; retries with error context (max 2)
7	Narrate	Converts tabular result into natural-language answer with sources
8	Output Guardrail	Tech name redaction ("DuckDB" → "our analytics engine"), PII scrubbing

6.2 MCP Servers (Model Context Protocol)

Four MCP tool servers extend the NLQ agent's capabilities:

MCP Server	Tools Provided
`market_data_server`	`get_stock_quote()`, `get_latest_signals()`, `get_insider_activity()`, `get_fii_dii_summary()`
`signal_server`	`search_signals_by_type()`, `get_signal_definition()`, `backtest_signal()`
`portfolio_server`	`get_portfolio()`, `compute_portfolio_return()`
`search_server`	`web_search()`, `search_news()`

7. WorldMonitor Global Intelligence

India's markets do not trade in isolation. AlphaStream integrates **WorldMonitor**, a global intelligence backbone, to provide cross-market context for every recommendation.

7.1 Global Data Feeds

Category	Instruments	India Impact
Global Indices	NIFTY, SENSEX, S&P 500, DOW, NASDAQ, FTSE	Correlation-based risk assessment
Crypto	BTC, ETH, SOL, XRP	Risk appetite proxy
Commodities	Gold, Crude Oil, Silver, Copper	Import cost impact on Indian cos
Currencies	INR/USD, DXY, EUR/USD	FX exposure for IT/Pharma
Fear & Greed	CNN Fear & Greed Index	Market sentiment gauge
Macro Signals	FRED: CPI, Yield Curve, Unemployment	Global recession risk
Geopolitical	India-focused geo-risk score	Border tensions, trade policy risk
VIX	CBOE Volatility Index	Global volatility regime

Figure 4: AlphaStream WorldMonitor Global Intelligence Dashboard

7.2 Integration Architecture

The `global_market_connector.py` reads WorldMonitor's shared JSON configurations (`stocks.json`, `commodities.json`, `sectors.json`) and fetches live data via yfinance and external APIs. Global signals are injected into the Decision Agent's context, enabling India-aware global reasoning: e.g., "Crude oil up 5% → OMCs like BPCL/IOC under pressure."

Nine dedicated API endpoints (`/api/global/*`) serve global data to the frontend's **Global Intel tab**, which features a Bloomberg-style ticker tape, sector heatmaps, Fear & Greed gauge, and macro signal panels.

8. Technology Stack

8.1 Backend

Component	Technology	Role
Streaming Engine	Pathway	Real-time RAG, <2s latency
Web Framework	FastAPI + WS + SSE	REST + real-time push
LLM	Gemini 2.0 Flash	All agent reasoning (Vertex AI)
Agent Framework	LangChain + LangGraph	Multi-agent orchestration + NLQ
Analytics DB	DuckDB	Pre-aggregated views, screener, Text2SQL
Vector Search	ChromaDB	Embedding-based article retrieval
Anomaly Detection	River ML (HST)	Online price/volume anomalies
Indian Data	NSE, BSE, Groww, NSDL	OHLCV, filings, FII/DII, fundamentals
Global Data	WorldMonitor, FRED	Indices, crypto, FX, macro
Package Manager	uv	Fast, reproducible dependencies

8.2 Frontend

Component	Technology	Role
Framework	React 19	UI framework
Build Tool	Vite 6	Dev server + bundling
Styling	Tailwind CSS 4	Utility-first CSS
Components	shadcn/ui	Accessible UI primitives
State	Zustand (persisted)	Client state + localStorage
Candlesticks	lightweight-charts 5.1	TradingView-style charting
Data Charts	Recharts 3.6	Bar, Pie, Radar, Line charts
Icons	Lucide React	Consistent iconography

9. API Reference

9.1 Market Intelligence (15 endpoints)

Endpoint	Method	Description
`/api/ohlcv/{ticker}`	GET	Candlestick + optional RSI/SMA
`/api/fundamentals/{ticker}`	GET	PE, PB, ROE, Div Yield, 52w H/L
`/api/radar`	GET	Top signals by Alpha Score
`/api/screener`	GET	Filter Nifty 50 (DuckDB view)
`/api/patterns/{ticker}`	GET	Chart pattern detection
`/api/backtest/{ticker}/{pat}`	GET	Historical backtest (5-year)
`/api/flows`	GET	FII/DII flows + streak detection
`/api/anomalies/{ticker}`	GET	River ML anomaly scores
`/api/filings/{ticker}`	GET	BSE corporate announcements
`/api/tickers`	GET	Nifty 50 universe
`/api/tickers/popular`	GET	Top 10 by market cap
`/api/news`	GET	Recent articles from DuckDB
`/api/portfolio`	POST	Set user portfolio holdings
`/api/portfolio/summary`	GET	Live P&L summary
`/api/insights`	GET	Ambient AI alerts

9.2 Global Market (9 endpoints)

Endpoint	Method	Description
`/api/global/indices`	GET	NIFTY, SENSEX, S&P 500, DOW, NASDAQ
`/api/global/commodities`	GET	Gold, Crude Oil, Silver, Copper
`/api/global/crypto`	GET	BTC, ETH, SOL, XRP
`/api/global/currencies`	GET	INR/USD, DXY, EUR/USD
`/api/global/sectors`	GET	12 US sector ETF returns
`/api/global/fear-greed`	GET	CNN Fear & Greed index
`/api/global/macro`	GET	FRED macro signals + verdict
`/api/global/vix`	GET	VIX volatility index
`/api/global/geo-risk`	GET	India geopolitical risk score

9.3 Core + Streaming (6 endpoints)

Endpoint	Method	Description
`/recommend`	POST	Full 13-agent recommendation
`/api/nlq`	POST	NLQ blocking response
`/api/nlq/stream`	POST/GET	NLQ with SSE streaming
`/insider/{ticker}`	GET	NSE SAST/PIT insider trading
`/report/{ticker}`	POST	PDF report generation
`/ws/stream/{ticker}`	WS	Real-time rec + market updates

10. Bloomberg-Style Terminal (Frontend)

The React 19 frontend presents a **5-tab Bloomberg-style terminal** with 25+ components:

#	Tab	Components
1	Overview	TradingView candlestick chart (RSI + SMA overlays), AI Recommendation Card (BUY/SELL/HOLD + reasoning), Stock Fundamentals (PE, PB, ROE from Groww), Insider Activity, Agent Radar, Anomaly Panel
2	Signals	Stock Screener (filter Nifty 50 by sector/direction/alpha), Opportunity Radar (signal cards with backtest win rates), FII/DII Flow Chart, Sector Heatmap
3	Global Intel	Bloomberg ticker tape, Global Markets Panel (indices, crypto, commodities), Fear & Greed Gauge, Macro Signal Panel (FRED), Commodity Strip, Geopolitical Risk Panel
4	Company	Corporate Filings (BSE, 7/30/90-day view), Articles List (sorted by threat level), Watchlist Panel (up to 20 tickers)
5	Portfolio	Portfolio Manager (add/remove holdings, real-time P&L), History Chart, Sector Allocation PieChart

Figure 5: AlphaStream Bloomberg-Style React 19 Frontend Terminal

The **NLQ Panel** ("Market ChatGPT Next Gen") is accessible from any tab via a floating button. It streams responses via SSE, shows the thought process (guardrail → route → SQL → narrate), and provides context-aware quick prompts that change based on the active tab.

11. Business Impact Model

11.1 Market Sizing (TAM / SAM / SOM)

Level	Segment	Size	Rationale
TAM	Total Indian equity participants	14 Cr	CDSL + NSDL demat accounts (2024)
SAM	Active traders researching daily	~3 Cr	≈20% trade monthly
SOM	5-year premium capture	1.5 L	0.5% of SAM (conservative)

Key assumption: SOM uses 0.5% of SAM, not TAM – anchored to paying users, not total accounts.

11.2 User Impact – Time Saved

Time Value Recovered Per User

1.75 hours/day × Rs. 500/hr × 250 trading days = $\mathbf{Rs.\ 2.19\ Lakh/user/year}$ in recovered productivity.

Task	Without AlphaStream	With AlphaStream
Daily research	2+ hours	15 minutes
Filing analysis	30 min/filing (manual)	Instant LLM classification
Insider trade monitoring	Manual NSE checks	Automated alerts
FII/DII tracking	Visit NSDL daily	Real-time flow analysis

11.3 User Impact – Alpha Generated

Signal Type	Detection	Win Rate	Avg 30d Return
RSI Divergence	Automated scan	60–78%	+3.26%
MACD Crossover	Real-time detect.	45–57%	+0.71%
Insider Cluster Buy	3+ insiders/30d	High (historical)	Varies
FII Buying Streak	5+ sessions	~72%	+3–5%

Conservative Alpha Estimate

2–3 actionable signals/month × 60% accuracy × Rs. 3,000 average gain = $\mathbf{Rs.\ 54,000/user/year}$ in additional alpha.

11.4 Risk Reduction

- Early warning on adverse signals prevents panic selling
- Portfolio-aware alerts: "Your IT holdings at risk from FII selling streak"
- Backtested confidence: "This pattern has worked 78% of the time on this stock"
- Estimated **15% reduction** in behavioral losses

11.5 ET Markets Revenue Projection

	Year 1	Year 3	Year 5
Premium users (Rs. 999/month)	50,000	2,00,000	5,00,000
Annual Revenue	**Rs. 59.9 Cr**	**Rs. 239.8 Cr**	**Rs. 599.4 Cr**

11.6 Cost Reduction for ET Markets

Cost Category	Current	With AlphaStream	Saving
Signal content (est.)	Rs. 20 Cr/yr (manual analysts)	LLM-generated, human-reviewed signal summaries	Rs. 5–10 Cr/yr (30% reduction)

Automated pipelines handle high-frequency, repeatable content (FII flows, insider alerts, technical scans). Humans focus on narrative and investigative content – higher value, lower volume.

11.7 Competitive Moat

1. **India-specific regulatory data** – NSE SAST/PIT and NSDL FII/DII feeds not available through generic global APIs.
2. **Backtested signals, not LLM speculation** – every signal carries a win rate grounded in NSE/BSE historical data.
3. **Streaming architecture** – Pathway incremental indexing vs. static dashboards that recompute on every load.
4. **Portfolio-aware alerts** – signals contextualised to each user's actual holdings.
5. **Compound improvement** – each interaction improves signal ranking; accuracy scales with user base.

11.8 Assumptions (Stated Explicitly)

1. Win rates based on backtesting against Nifty 50 stocks with 3–5 year data
2. Average return estimates use median, not mean (to reduce outlier effect)
3. Time savings assumes basic digital literacy and daily market participation
4. Premium conversion rate of 0.3% is conservative (industry average 0.5–1%)
5. Back-of-envelope calculations – detailed validation would require A/B testing
6. SOM (1.5L users) = 0.5% of SAM (3 Cr active traders) – conservative for 5-year horizon
7. Revenue projections assume no price increase over 5 years (understates upside)
8. Cost reduction estimate assumes research content budget of ∼Rs. 20 Cr/year
9. Year 3/5 projections assume compounding growth of ∼100%/150% from Year 1

12. Demonstration & Verification

12.1 Live Demo Flow

1. **Login** – Open dashboard at `http://localhost:5173`; login as judge
2. **Overview Tab** – RELIANCE loads by default: candlestick chart, AI recommendation, fundamentals, insider activity, anomaly detection
3. **Signals Tab** – Stock Screener filters Nifty 50; Opportunity Radar shows top signals
4. **Global Intel** – Bloomberg ticker tape, crypto, FX, Fear & Greed, macro signals
5. **NLQ Chat** – Ask "Which stocks have FII buying streaks?" – grounded Text2SQL answer
6. **Real-time** – Inject breaking news via API; watch recommendation update in $<$2s

12.2 Expected Latencies

Operation	Latency
Article ingestion to DuckDB	<100 ms
Pathway RAG indexing	<200 ms
13-agent recommendation	∼1.2 s (LLM-bound)
WebSocket delivery to browser	<50 ms
Total: news → UI update	**<2 seconds**
Adaptive RAG token savings	∼40% vs fixed-k
PDF report generation	∼15 s

13. Setup & Quick Start

13.1 Prerequisites

- Python 3.11+ and uv package manager
- Node.js 18+ (frontend)
- Google Cloud account with Vertex AI (Gemini 2.0 Flash)
- `NEWS_API_KEY` from newsapi.org (for live news)
- 4 GB+ RAM recommended

13.2 Three-Command Setup

```
# 1. Clone and install backend
git clone https://github.com/wildcraft958/AlphaStream_India.git
cd AlphaStream_India/backend
cp .env.example .env   # Add API keys
uv sync && uv run python -m src.data.market_schema

# 2. Start backend (cold start ~75s)
./start.sh   # OR: uv run uvicorn src.api.app:app --port 8000

# 3. Start frontend (new terminal)
cd ../frontend && npm install && npm run dev
# Open http://localhost:5173
```

13.3 Key Environment Variables

Variable	Purpose
`GOOGLE_APPLICATION_CREDENTIALS`	Path to GCP service-account JSON
`GCP_PROJECT_ID`	GCP project ID for Vertex AI
`NEWS_API_KEY`	NewsAPI key for live article ingestion
`ENABLE_PATHWAY`	`true` = Pathway RAG; `false` = sync fallback
`GROWW_API_TOKEN`	Groww fundamentals (PE, PB, ROE)
`FRED_API_KEY`	FRED macro signals (CPI, yield curve)

Appendix: Repository Structure

	Directory / File	Purpose
BACKEND	`backend/src/agents/`	13 AI agents (Sentiment, Technical, Risk, ...) + NLQ pipeline + MCP servers
	`backend/src/connectors/`	Data source adapters: NSE, BSE, Groww, WorldMonitor, FII/DII
	`backend/src/pipeline/`	Pathway streaming engine + Adaptive RAG + Fusion Engine
	`backend/src/api/`	FastAPI application: market, NLQ, global routers
	`backend/src/data/`	DuckDB schema definitions + Nifty 50 seed data
	`backend/start.sh`	One-command startup (respects `ENABLE_PATHWAY`)
FE	`frontend/src/components/`	25+ Bloomberg terminal components (charts, screener, NLQ)
	`frontend/src/store/`	Zustand state management + localStorage persistence
	`worldmonitor/`	Global intelligence: 3D globe, 86 panels, 435+ news feeds
DOCS	`docs/*.png`	Architecture diagrams (system, agents, herd-of-knowledge)
	`docs/impact_model.md`	TAM/SAM/SOM + revenue projections
	`docs/submission.tex`	This document (LaTeX source)
	`demo_pipeline.py`	8-step automated demo with JSON proof generation
	`README.md`	Project overview + full API reference (24 endpoints)

Full source: `https://github.com/wildcraft958/AlphaStream_India`